EE 446 TinyML - Homework 1 Discussion Answers

Name: _____

Typed discussion responses for Problem 1(e) and Problem 2. Programming outputs are in the completed notebook.

Problem 1(e): Further Model Size Reduction

In the notebook run, the baseline float32 TFLite model reached 0.9815 test accuracy with a size of 14.07 KB. Among parts (b), (c), and (d), the smallest prior model was the full-integer int8 quantized baseline at 5.74 KB, also with 0.9815 test accuracy. Dynamic range quantization and float16 quantization preserved accuracy but were larger at 8.17 KB and 8.95 KB. The pruned model was 8.06 KB and had lower accuracy in this run, while the student model from knowledge distillation was 6.10 KB with 0.9815 accuracy.

Model	Size (KB)	Test accuracy
Float32 baseline	14.07	0.9815
Full int8 quantized baseline	5.74	0.9815
Float16 quantized baseline	8.95	0.9815
Dynamic range quantized baseline	8.17	0.9815
Pruned model	8.06	0.9259
KD student	6.10	0.9815
Compact KD + int8 model	3.34	0.9815

My strategy was to combine two ideas: make the student network smaller than the part (d) student and then apply full integer quantization. I used a compact 24-neuron / 16-neuron hidden-layer student, trained with a weighted hard-label/teacher-soft-label distillation loss, then converted it to a fully int8 TFLite model. This produced model_compact_kd_int8.tflite at 3.34 KB. Compared with the smallest prior model, the compact model is 2.40 KB smaller while keeping the same measured test accuracy. The biggest size reduction came from reducing the dense-layer parameter count before quantization; int8 quantization then stored the remaining weights much more compactly. Because the Wine dataset is small and fairly separable after standardization, the compact model retained essentially the same classification performance on this split.

Problem 2.1: Free-Tier Learning Blocks

The main free-tier learning block categories are supervised learning and anomaly detection. For supervised learning, Edge Impulse provides Classification (Keras), Regression (Keras), Image Classification using transfer learning, Keyword Spotting using transfer learning, and Object Detection using either FOMO or MobileNetV2 SSD FPN. Example models include a motion classifier from accelerometer features, a regression model that predicts a continuous sensor value, an image classifier based on MobileNet transfer learning, a small keyword spotter, and a FOMO object detector. For anomaly detection, the available blocks include K-means anomaly detection, GMM anomaly detection, and FOMO-AD visual anomaly detection. Example models include a vibration anomaly detector for machinery and a visual defect detector for product inspection. I did not find a separate self-supervised learning block in the free-tier Studio learning-block list. Classical ML appears in the general learning-block list, but the current Classical ML page marks that feature as Enterprise-only, so I would not count it as a free-tier block.

Problem 2.2: Classifier Layers

Layer type	Purpose
Input	Receives feature tensors with the required shape.
Dense / fully connected	Learns nonlinear combinations of all previous-layer features; common for tabular or flattened features.

Reshape	Changes tensor shape without changing values so data can be passed to sequence or image-style layers.
Flatten	Converts multidimensional outputs into a 1D vector before dense/output layers.
Dropout	Regularizes training by randomly disabling units during updates to reduce overfitting.
1D convolution	Extracts local patterns from sequential/time-series/audio-derived features.
1D pooling	Downsamples 1D feature maps, usually with max or average pooling, to reduce computation and add invaria
2D convolution	Extracts spatial patterns from image-like data or spectrograms.
2D pooling	Downsamples 2D feature maps to reduce spatial size and computation.
Batch normalization	Normalizes intermediate activations to stabilize and often speed up training.
Output	Produces the final class probabilities or regression output; classification commonly ends with softmax.

Problem 2.3: Deployment Compression / Optimization Options

The Deployment page exposes model-version choices and EON Compiler optimizations. The model-version choice lets the user deploy either an unoptimized float32 model or a quantized int8 model; the int8 model is the primary free-tier model compression option because it stores weights and activations in 8-bit integer form. Edge Impulse also offers the EON Compiler, which converts the model into optimized C++ for edge inference and can reduce RAM, flash, and latency while preserving accuracy. The EON Compiler RAM-optimized option is listed as Enterprise-only, so I would not count that as free-tier.

Problem 2.4: Synthetic Data Modalities and Importance

The Synthetic Data tab lists built-in blocks for image, audio/keyword spotting, and time-series data. For image data, the provided methods are the DALL-E Image Generation Block and the FLUX Pro Image Generation Block. For audio/keyword spotting, the provided method is the Whisper Keyword Spotting Generation Block, which generates speech examples from a keyword prompt. Time-series Data Augmentation is also listed, but the current documentation marks it as Enterprise-only, so I would not treat it as a free-tier option. Synthetic data is useful because it can fill gaps in the dataset, increase coverage of rare cases, reduce manual collection cost, rebalance classes, and improve robustness. It should still be validated against real data because generated samples can introduce distribution shift or unrealistic artifacts.

References

Edge Impulse Learning Blocks: https://docs.edgeimpulse.com/studio/projects/learning-blocks

Edge Impulse Layers: https://docs.edgeimpulse.com/knowledge/concepts/machine-learning/neural-networks/layers

Edge Impulse Keras Visual Layer Type API enum:
https://docs.edgeimpulse.com/tools/libraries/api-bindings/studio/python/edgeimpulse_api/models/keras_visual_layer_type

Edge Impulse Deployment: https://docs.edgeimpulse.com/studio/projects/deployment

Edge Impulse EON Compiler: https://docs.edgeimpulse.com/studio/projects/deployment/eon-compiler

Edge Impulse Synthetic Data: https://docs.edgeimpulse.com/studio/projects/data-acquisition/synthetic-data

Edge Impulse Classical ML: https://docs.edgeimpulse.com/studio/projects/learning-blocks/blocks/classical-ml